

ECE 455 B04 — Real-Time Computer Systems

Project 1: Traffic Light System

Student Manual

STM32F4 Discovery Board | FreeRTOS v9 | GPIO, ADC & Shift Registers

University of Victoria — Department of Electrical and Computer Engineering

Created by:

Amir Sepahi

Table of Contents

1. Project Overview
2. Hardware Overview: STM32F4 Discovery Board
3. GPIO Setup — General Purpose Input/Output
 - 3.1 Enabling the Clock
 - 3.2 Configuring GPIO Pins (GPIO_Init)
 - 3.3 GPIO Parameters Explained
 - 3.4 Setting and Resetting Pins
4. ADC Setup — Analog-to-Digital Converter
 - 4.1 Enabling Clocks for ADC
 - 4.2 Configuring the ADC GPIO Pin
 - 4.3 ADC Initialization Parameters
 - 4.4 Performing an ADC Conversion
5. Shift Registers — Serial-to-Parallel Conversion
 - 5.1 How Shift Registers Work
 - 5.2 GPIO Pins for the Shift Register
 - 5.3 Pushing Data and Resetting
6. FreeRTOS Fundamentals
 - 6.1 Tasks
 - 6.2 Queues
 - 6.3 Software Timers
 - 6.4 Delays and Timing
7. Project Architecture: Tasks, Queues & Data Flow
 - 7.1 System Block Diagram
 - 7.2 Task Descriptions
 - 7.3 Queue Definitions
 - 7.4 Software Timer Definitions
8. Step-by-Step Implementation Roadmap
9. Pin Mapping Reference
10. Common Pitfalls and Debugging Tips

1. Project Overview

In this project you will build a traffic light controller running on an STM32F4 Discovery board using FreeRTOS. The system simulates a single-lane intersection with the following behavior:

Traffic lights (red, amber, green LEDs) cycle through their phases with configurable timing. A potentiometer connected through an ADC controls the traffic flow rate — turning the knob adjusts both how frequently new cars appear and how long each light phase lasts. Cars are represented by LEDs driven through daisy-chained shift registers. When the light is green, cars flow through the intersection. When the light turns amber or red, cars accumulate (queue up) before the intersection.

The entire system is organized into four FreeRTOS tasks communicating through five queues, with three software timers controlling light phase durations.

2. Hardware Overview: STM32F4 Discovery Board

The STM32F4 Discovery board features an ARM Cortex-M4 processor (STM32F407VG) running at up to 168 MHz. For this project you will use three peripherals:

GPIO (General Purpose Input/Output)

Digital pins that can be configured as inputs or outputs. You will use GPIO Port C for all connections: traffic light LEDs on pins PC0–PC2, and shift register control lines on pins PC6–PC8.

ADC (Analog-to-Digital Converter)

Converts the analog voltage from a potentiometer into a 12-bit digital value (0–4095). The potentiometer is connected to PC3, which maps to ADC1 Channel 13.

Shift Registers (74HC595)

Three daisy-chained serial-to-parallel shift register ICs provide 19 LED outputs using only 3 GPIO pins (data, clock, reset). Each IC provides 8 output lines, giving 24 total (of which 19 are used to represent cars).

3. GPIO Setup — General Purpose Input/Output

Every GPIO operation on the STM32F4 follows three steps: enable the clock for the GPIO port, configure the pin parameters using `GPIO_Init`, and then set or reset individual pins during runtime. Understanding each step and its parameters is critical.

3.1 Enabling the Clock

Before any peripheral can be used, its clock must be enabled. The STM32F4 has two main bus types:

Bus	Full Name	Typical Peripherals	Enable Function
AHB1	Advanced High-performance Bus 1	GPIO ports (A, B, C, D, E, ...)	<code>RCC_AHB1PeriphClockCmd()</code>
APB1	Advanced Peripheral Bus 1	UART, SPI, I2C, basic timers	<code>RCC_APB1PeriphClockCmd()</code>
APB2	Advanced Peripheral Bus 2	ADC, USART1, SPI1, advanced timers	<code>RCC_APB2PeriphClockCmd()</code>

Since all our GPIO pins are on Port C, the clock enable call is:

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
```

Why Enable the Clock?

To save power, the STM32 keeps peripheral clocks disabled by default. Attempting to configure or use a peripheral without enabling its clock first will silently fail — no error, just nothing works. This is the single most common mistake students make.

3.2 Configuring GPIO Pins (`GPIO_Init`)

`GPIO_Init` takes two arguments: the GPIO port (e.g., `GPIOC`) and a pointer to a configuration structure of type `GPIO_InitTypeDef`. You fill in five fields of this structure before calling the function.

Here is a complete example that configures PC0 as a push-pull output:

```
GPIO_InitTypeDef GPIO_InitStructure;

// Enable clock for Port C
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);

// Configure pin(s)
GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_0;
GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_OUT;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_PuPd  = GPIO_PuPd_UP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

GPIO_Init(GPIOC, &GPIO_InitStructure);
```

3.3 GPIO Parameters Explained

The following table describes every field of the `GPIO_InitTypeDef` structure, what each setting means, and which value to use in this project.

Parameter	Options	Meaning	Project Setting
GPIO_Pin	GPIO_Pin_0 through GPIO_Pin_15 (can OR multiple)	Selects which physical pin(s) on the port to configure. You can combine pins with the bitwise OR operator to configure several at once.	Traffic lights: Pin_0 Pin_1 Pin_2; Shift reg: Pin_6 Pin_7 Pin_8
GPIO_Mode	GPIO_Mode_OUT, GPIO_Mode_IN, GPIO_Mode_AF, GPIO_Mode_AN	OUT = digital output. IN = digital input. AF = alternate function (UART, SPI, etc.). AN = analog (required for ADC input).	OUT for LEDs and shift register; AN for potentiometer (ADC pin)
GPIO_OType	GPIO_OType_PP (Push-Pull), GPIO_OType_OD (Open-Drain)	Push-Pull: pin drives both HIGH and LOW actively. Open-Drain: pin only pulls LOW; needs external pull-up for HIGH. Push-Pull is the standard choice for driving LEDs.	GPIO_OType_PP for all output pins
GPIO_PuPd	GPIO_PuPd_NOPULL, GPIO_PuPd_UP, GPIO_PuPd_DOWN	Activates an internal pull-up or pull-down resistor. UP ensures pin reads HIGH when floating. NOPULL is required for ADC analog inputs (pull resistors would distort the analog reading).	GPIO_PuPd_UP for outputs; GPIO_PuPd_NOPULL for ADC pin
GPIO_Speed	GPIO_Speed_2MHz, GPIO_Speed_25MHz, GPIO_Speed_50MHz, GPIO_Speed_100MHz	Sets the slew rate (how fast the output voltage transitions). Higher speed means faster edges but more electromagnetic noise and power. 50 MHz is a safe general choice.	GPIO_Speed_50MHz

Combining Pins

You can initialize multiple pins at once using bitwise OR: `GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2`; This configures PC0, PC1, and PC2 simultaneously with the same settings. This is both more efficient and cleaner than calling `GPIO_Init` three times.

3.4 Setting and Resetting Pins

Once configured, you control output pins at runtime with two functions:

Function	Effect	Example
GPIO_SetBits(GPIOx, Pin)	Drives the pin HIGH (3.3V)	GPIO_SetBits(GPIOC, GPIO_Pin_0); // Red LED ON
GPIO_ResetBits(GPIOx, Pin)	Drives the pin LOW (0V)	GPIO_ResetBits(GPIOC, GPIO_Pin_0); // Red LED OFF

4. ADC Setup — Analog-to-Digital Converter

The ADC reads the voltage from the traffic-rate potentiometer and converts it to a digital value between 0 and 4095 (12-bit resolution). This value is then used by the system to control how frequently new cars appear and how long the green and red light phases last.

4.1 Enabling Clocks for ADC

The ADC requires two clocks to be enabled: the GPIO clock for the analog input pin, and the ADC peripheral clock itself. Note that ADC1 is on the APB2 bus, not AHB1.

```
// ADC1 is on APB2
RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
// The GPIO pin (PC3) still needs AHB1
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
```

4.2 Configuring the ADC GPIO Pin

The pin connected to the potentiometer must be configured in Analog mode with no pull-up/pull-down resistor. Any internal resistor would form a voltage divider with the potentiometer and distort the reading.

```
GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AN;      // Analog mode
GPIO_InitStructure.GPIO_Pin  = GPIO_Pin_3;       // PC3
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; // No pull resistor
GPIO_Init(GPIOC, &GPIO_InitStructure);
```

Important

You do not need to set GPIO_OType or GPIO_Speed for analog pins. These fields only apply to digital output modes.

4.3 ADC Initialization Parameters

After configuring the GPIO pin, you configure the ADC peripheral itself through the ADC_InitTypeDef structure. Each parameter controls a specific aspect of the conversion.

Parameter	Value Used	What It Means
ADC_ContinuousConvMode	DISABLE	When DISABLED, the ADC performs one conversion per software trigger and stops. When ENABLED, it continuously converts in a loop. We want single-shot since we read on demand.
ADC_DataAlign	ADC_DataAlign_Right	The 12-bit result is placed in the lower 12 bits of a 16-bit register (bits 11:0). Left alignment would place it in bits 15:4. Right alignment makes reading the value straightforward.
ADC_ExternalTrigConv	DISABLE	Disables hardware trigger sources (timers, external pins). We trigger conversions

		manually in software using <code>ADC_SoftwareStartConv()</code> .
<code>ADC_ExternalTrigConvEdge</code>	<code>ADC_ExternalTrigConvEdge_None</code>	No trigger edge detection since we are not using an external trigger.
<code>ADC_NbrOfConversion</code>	1	Number of channels to convert in a sequence. We only read one potentiometer, so this is 1.
<code>ADC_Resolution</code>	<code>ADC_Resolution_12b</code>	The precision of the conversion. 12-bit gives values 0–4095. Other options are 10-bit (0–1023), 8-bit (0–255), and 6-bit (0–63).
<code>ADC_ScanConvMode</code>	<code>DISABLE</code>	Scan mode converts multiple channels in sequence. Since we only have one channel, this must be disabled.

After filling the structure, enable the ADC and configure the input channel:

```
ADC_Init(ADC1, &ADC_InitStruct);
ADC_Cmd(ADC1, ENABLE); // Power on ADC1

// Configure Channel 13 (PC3), rank 1, 84-cycle sample time
ADC-RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_84Cycles);
```

Channel Mapping

PC3 corresponds to ADC Channel 13. This mapping is fixed in hardware — see Table 10 in the STM32F407 datasheet. The sample time (84 cycles) determines how long the ADC samples the input voltage before converting. Longer sample times give more accurate readings for high-impedance sources like potentiometers.

4.4 Performing an ADC Conversion

Reading the potentiometer is a three-step process: start the conversion, wait for completion, and retrieve the result.

```
static uint16_t ADC_Start_Conversion() {
    uint16_t converted_data;
    // 1. Trigger the conversion
    ADC_SoftwareStartConv(ADC1);
    // 2. Wait for End-of-Conversion flag
    while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
    // 3. Read the result (0–4095)
    converted_data = ADC_GetConversionValue(ADC1);
    return converted_data;
}
```

5. Shift Registers — Serial-to-Parallel Conversion

5.1 How Shift Registers Work

A 74HC595 shift register converts serial data (one bit at a time) into 8 parallel outputs. By daisy-chaining three ICs, you get 24 outputs using only 3 control pins. In this project, 19 of these outputs drive LEDs that represent cars on the road.

The basic operation is: (1) set the Data pin to 0 or 1, (2) pulse the Clock pin to shift that bit into the register. Each clock pulse pushes the existing bits one position forward. After shifting all 19 bits, the LEDs reflect the car positions.

5.2 GPIO Pins for the Shift Register

Signal	GPIO Pin	Purpose
Data	PC6 (GPIO_Pin_6)	The bit value to shift in (HIGH = car present, LOW = no car)
Clock	PC7 (GPIO_Pin_7)	Rising edge shifts the data bit into the register
Reset	PC8 (GPIO_Pin_8)	Active-LOW: pulling this low clears all outputs to 0

6. FreeRTOS Fundamentals

FreeRTOS is a real-time operating system kernel that provides multitasking on microcontrollers. Your project uses four core FreeRTOS mechanisms: tasks, queues, software timers, and delays.

6.1 Tasks

A task is an independent thread of execution with its own stack. Each task runs in an infinite `while(1)` loop and the FreeRTOS scheduler switches between tasks based on their priorities. Tasks are created before the scheduler starts.

```
// Task function signature
static void MyTask(void *pvParameters) {
    while(1) {
        // Task logic here
        vTaskDelay(pdMS_TO_TICKS(500)); // Yield for 500ms
    }
}

// Creating the task
xTaskCreate(
    MyTask,                // Function pointer
    "TaskName",            // Debug name (string)
    configMINIMAL_STACK_SIZE, // Stack size in words
    NULL,                  // Parameter passed to task
    1,                     // Priority (higher = more urgent)
    NULL                   // Task handle (optional)
);
```

Parameter	Description
Function pointer	The C function that implements the task. Must have signature <code>void func(void*)</code>
Task name	A human-readable string for debugging. Visible in the kernel-aware debugger.
Stack size	Amount of memory for the task's local variables and call stack. <code>configMINIMAL_STACK_SIZE</code> is defined in <code>FreeRTOSConfig.h</code> .
Parameters	A void pointer passed to the task function. Use <code>NULL</code> if no parameters needed.
Priority	Higher numbers mean higher priority. Equal-priority tasks share CPU time using round-robin scheduling.
Task handle	Optional output handle for later referencing the task (e.g., to delete or suspend it). <code>NULL</code> if not needed.

6.2 Queues

Queues are the primary inter-task communication mechanism in FreeRTOS. They are thread-safe FIFO buffers that allow one task to send data and another to receive it, with optional blocking (waiting) if the queue is full or empty.

```
// Create a queue that holds 5 items, each sizeof(uint32_t) bytes
xQueueHandle myQueue = xQueueCreate(5, sizeof(uint32_t));
```

```
// Register for kernel-aware debugging
xQueueAddToRegistry(myQueue, "MyQueueName");

// Sending (from one task)
uint32_t value = 42;
xQueueSend(myQueue, &value, pdMS_TO_TICKS(1000)); // Block up to 1s

// Receiving (from another task)
uint32_t received;
if (xQueueReceive(myQueue, &received, pdMS_TO_TICKS(1000))) {
    // Successfully received
}
```

Function	Parameters	Behavior
xQueueCreate(length, itemSize)	length: max items in queue; itemSize: bytes per item	Allocates the queue. Returns a handle (NULL on failure).
xQueueSend(queue, &item, timeout)	queue: handle; item: data to copy; timeout: max ticks to wait	Copies item into queue. Blocks if full until timeout. Returns pdTRUE on success.
xQueueReceive(queue, &buffer, timeout)	queue: handle; buffer: where to copy; timeout: max ticks	Copies oldest item out of queue into buffer. Blocks if empty. Returns pdTRUE on success.

Key Concept: Blocking vs Non-Blocking

When you pass a timeout of 0, the call returns immediately (non-blocking). When you pass portMAX_DELAY, the task blocks indefinitely until an item is available. Any other value specifies a maximum wait in ticks. Use pdMS_TO_TICKS() to convert milliseconds.

6.3 Software Timers

Software timers execute a callback function after a specified period. They are managed by the FreeRTOS timer daemon task, not by hardware timer peripherals. In this project, each traffic light phase (red, amber, green) uses its own software timer to signal when that phase has expired.

```
// Create a one-shot timer (pdFALSE = does not auto-reload)
TimerHandle_t myTimer = xTimerCreate(
    "TimerName",           // Debug name
    pdMS_TO_TICKS(5000),   // Period: 5 seconds
    pdFALSE,               // One-shot (pdTRUE = auto-reload)
    0,                    // Timer ID (user-defined)
    myCallbackFunction     // Called when timer expires
);

// Start the timer
xTimerStart(myTimer, 0);

// Change period dynamically (also starts the timer)
xTimerChangePeriod(myTimer, pdMS_TO_TICKS(3000), 0);

// Callback function signature
static void myCallbackFunction(TimerHandle_t xTimer) {
```

```
    // Runs in timer daemon context, keep it short!  
}
```

6.4 Delays and Timing

FreeRTOS delays release the CPU to other tasks while waiting:

Function	Behavior
<code>vTaskDelay(ticks)</code>	Delays the calling task for the specified number of ticks relative to when the call is made. Use <code>pdMS_TO_TICKS(ms)</code> to convert from milliseconds.
<code>vTaskDelayUntil(&prevWake, ticks)</code>	Delays until an absolute tick count, providing more precise periodic timing than <code>vTaskDelay</code> .
<code>pdMS_TO_TICKS(ms)</code>	Macro that converts milliseconds to RTOS ticks based on the configured tick rate (typically 1 kHz = 1 ms/tick).

7. Project Architecture: Tasks, Queues & Data Flow

The system uses four tasks and five queues. Understanding how data flows between them is essential to implementing the project correctly.

7.1 System Block Diagram

The data flow is: Adjust_Traffic reads the potentiometer and sends the value to both Traffic_light (for timing) and Create_Traffic (for car generation rate). Traffic_light manages light phases using software timers and sends the current light state to Create_Traffic (so it knows whether to accumulate or flow cars). Create_Traffic computes the new car pattern and sends it to Display_Traffic, which drives the shift register LEDs.

7.2 Task Descriptions

Task	Function	Role	Priority
Adjust_Traffic	Adjust_Traffic()	Reads the potentiometer via ADC. Sends the raw value (0–4095) to two queues: one for adjusting light timing, one for adjusting car creation rate.	1
Traffic_light	Traffic_light()	Manages the traffic light state machine: green → amber → red → green. Uses software timers for each phase. Adjusts green and red durations based on potentiometer value. Sends current light state to Create_Traffic.	1
Create_Traffic	Create_Traffic()	Generates the car pattern as a 19-bit integer. When light is green, cars shift through (moving past intersection). When amber/red, cars accumulate before the intersection. New cars appear randomly based on the potentiometer-controlled rate.	1
Display_Traffic	Display_Traffic()	Receives the 19-bit car pattern and drives the shift register to display it on LEDs. Clears all LEDs first, then shifts in each bit from MSB to LSB.	1

7.3 Queue Definitions

The project defines five queues. Each carries a specific type of data between specific tasks.

Queue Handle	Item Type	From → To	Purpose
xTrafficFlowQueue_handle	uint32_t	Create_Traffic → Display_Traffic	The 19-bit car pattern to display on LEDs
xTrafficNewLightQueue_handle	uint8_t	Traffic_light → Create_Traffic	Notifies Create_Traffic of a new light state (green, amber, or red)
xTrafficExpiredLightQueue_handle	uint8_t	Timer callback → Traffic_light	Notifies Traffic_light that the current phase timer has expired
xTrafficAdjustRateQueue_handle	uint16_t	Adjust_Traffic → Create_Traffic	Potentiometer value to control car generation frequency

xTrafficLightAdjustTimingQueue_handle	uint16_t	Adjust_Traffic → Traffic_light	Potentiometer value to control light phase durations
---------------------------------------	----------	-----------------------------------	---

All queues are created with a length of 5 (mainQUEUE_LENGTH). This means each queue can buffer up to 5 unread messages before a send operation would block.

7.4 Software Timer Definitions

Timer Handle	Default Period	Type	What Happens on Expiry
xGreenLightTimmer_handle	10,000 ms (10 s)	One-shot	Sends "green expired" to xTrafficExpiredLightQueue, causing transition to amber
xAmberLightTimmer_handle	2,000 ms (2 s)	One-shot	Sends "amber expired" to xTrafficExpiredLightQueue, causing transition to red
xRedLightTimmer_handle	7,000 ms (7 s)	One-shot	Sends "red expired" to xTrafficExpiredLightQueue, causing transition to green

Dynamic Timing

The green and red light durations are adjusted based on the potentiometer value using xTimerChangePeriod(). Higher potentiometer values (more traffic) increase green time and decrease red time. The amber duration stays fixed at 2 seconds.

8. Step-by-Step Implementation Roadmap

Follow these steps in order. Test each step before moving to the next using the `traffic_light_system` monitoring application and `printf()` through SWV (Serial Wire Viewer).

Step 1: Set Up the Project

Start with the Project 0 template code. Familiarize yourself with the FreeRTOS task, queue, and timer APIs by reading the existing demo code. Make sure you can build and flash to the board.

Step 2: Initialize GPIO for Traffic Lights

Write `GPIO_Init_Traffic_Light()` to configure PC0 (red), PC1 (amber), PC2 (green) as push-pull outputs. Test by manually calling `GPIO_SetBits/ResetBits` in `main()` before the scheduler starts — verify each LED lights up individually.

Step 3: Initialize GPIO for Shift Registers

Write `GPIO_Init_Shift_Register()` to configure PC6 (data), PC7 (clock), PC8 (reset) as push-pull outputs. Test by manually shifting a known pattern (e.g., alternating 1s and 0s) and verifying the car LEDs.

Step 4: Initialize ADC for Potentiometer

Write `Init_ADC()` to configure PC3 as an analog input and set up ADC1. Test by reading the value in a loop and printing it — turning the potentiometer should give values from 0 to about 4095.

Step 5: Create Queues

Define all five queue handles as global variables. Create them in `main()` using `xQueueCreate()` with appropriate lengths and item sizes. Register each with `vQueueAddToRegistry()` for debugging.

Step 6: Implement `Adjust_Traffic` Task

This is the simplest task. Read the ADC, send the value to both `xTrafficLightAdjustTimingQueue` and `xTrafficAdjustRateQueue`. Add a `vTaskDelay` to control how often the knob is sampled (about 1 second).

Step 7: Implement `Traffic_light` Task

Create three software timers (one per light phase). Implement the state machine: when a timer expires, transition to the next light. Use `xTimerChangePeriod` to adjust green and red durations based on the potentiometer value received from the queue.

Step 8: Implement `Create_Traffic` Task

Maintain a 19-bit integer representing the road. When the light is green, left-shift the pattern (cars move forward). When amber/red, use the `accumulate_traffic()` function to pile up cars before the intersection. Randomly add new cars based on the traffic rate.

Step 9: Implement Display_Traffic Task

Receive the car pattern from the queue, clear the shift register, then loop through all 19 bits from MSB to LSB, calling `add_car()` for each position.

Step 10: Integration and Testing

Start the scheduler with `vTaskStartScheduler()`. Use the `traffic_light_system` Python application to visually verify behavior. Test edge cases: potentiometer at minimum (low traffic), maximum (high traffic), and transitions between light phases.

9. Pin Mapping Reference

Function	Pin	GPIO Mode	Direction	Notes
Red LED	PC0	Output (PP)	Out	GPIO_Pin_0
Amber LED	PC1	Output (PP)	Out	GPIO_Pin_1
Green LED	PC2	Output (PP)	Out	GPIO_Pin_2
Potentiometer (ADC)	PC3	Analog (AN)	In	ADC1 Channel 13
Shift Reg. Data	PC6	Output (PP)	Out	GPIO_Pin_6
Shift Reg. Clock	PC7	Output (PP)	Out	GPIO_Pin_7
Shift Reg. Reset	PC8	Output (PP)	Out	GPIO_Pin_8, Active LOW

Defined Constants

```

#define red_light          GPIO_Pin_0
#define amber_light       GPIO_Pin_1
#define green_light       GPIO_Pin_2
#define Shift_Register_Data GPIO_Pin_6
#define Shift_Register_Clock GPIO_Pin_7
#define Shift_Register_Reset GPIO_Pin_8

#define green_light_time   10000 // ms
#define amber_light_time   2000  // ms
#define red_light_time     7000  // ms
#define lane_capacity      19     // number of LED positions
#define mainQUEUE_LENGTH  5       // queue depth

```


10. Common Pitfalls and Debugging Tips

Forgetting to enable the clock

This is the number one source of silent failures. If a peripheral does not respond at all, check that you called the correct `RCC_AXnPeriphClockCmd()` function. GPIO uses AHB1; ADC uses APB2.

Using the wrong ADC channel

PC3 maps to ADC1 Channel 13, not Channel 3. The channel number is determined by the hardware pin mapping, not the pin number. Always check the datasheet.

Queue full or empty warnings

If you see "queue full" messages, the receiving task is not reading fast enough, or the timeout is too short. If "queue empty," the sending task may not be running. Check task priorities and delays.

Shift register data appears reversed

The shift register displays bits in the order they were shifted in. The first bit pushed ends up at the far end. Make sure you iterate from MSB to LSB (bit 18 down to bit 0) when displaying.

Stack overflow

If the board crashes or resets, you may have insufficient stack. Try increasing `configMINIMAL_STACK_SIZE` in `FreeRTOSConfig.h`. Enable `configCHECK_FOR_STACK_OVERFLOW` for debugging.

Not using `pdMS_TO_TICKS()`

FreeRTOS delays and timer periods are in ticks, not milliseconds. Always wrap millisecond values with `pdMS_TO_TICKS()`. Forgetting this makes timings wrong by a factor of the tick rate.

Blocking in timer callbacks

Software timer callbacks run in the timer daemon task context. Never use blocking calls (like `xQueueSend` with a long timeout) inside a callback. Use a timeout of 0.

Potentiometer reading fluctuates

ADC readings naturally have some noise. Consider averaging multiple readings or adding hysteresis if the traffic rate jumps around erratically.

Printf not working

Make sure SWV (Serial Wire Viewer) is configured in your debug settings. Printf output goes through the ITM trace port, not a UART.